Federated Learning Across Simulated Hospitals

Technical report. Big Data and Responsible AI Engineering.

Team: Sunidhi Shekar and Sonia Sharma

Abstract

Five simulated hospitals can train a shared 30-day readmission model with federated learning at a small, measurable cost in accuracy, with no detectable change in cross-site fairness, and with a clear benefit to every hospital over training alone. We split the public UCI Diabetes 130-US Hospitals dataset into five non-IID sites and compare three approaches with the same model and training budget: centralized training on pooled data, federated training (Flower 1.42, FedAvg, 8 rounds), and local-only training at each hospital.
Over five seeds, federated training reaches a global ROC-AUC of 0.6696 ± 0.0012 against 0.6777 ± 0.0023 for centralized training, a paired difference of −0.008 (95% CI −0.013 to −0.003). The best-to-worst hospital gap does not differ significantly (0.090 against 0.080, p = 0.75), although hospital 2 loses 0.025 AUC under federation. On patients from all hospitals, the federated model beats every hospital's local model by 0.005 to 0.113 AUC. The costs are communication (4.5 MB of weights; 12.3 MB measured on the network) and time (34 s against 5.8 s). We also deploy the system as six isolated Docker containers, verify that no hospital can reach another's data, and reproduce the single-process results to within 0.0006 ROC-AUC per seed.

Introduction

Hospitals hold the data needed to predict readmissions, but privacy rules and institutional policy usually stop them from pooling patient records. In federated learning, each hospital trains a shared model on its own data and only model weights travel to a central server, which averages them into one global model.
Hospitals differ in size, patient mix and readmission rate, so their data is not identically distributed (non-IID). Averaging models trained on different populations can cost accuracy, and a global average can hide a hospital that does worse. We ask four questions:

1. **Accuracy.** How much performance does federated training give up compared with centralized training, and how much does it gain over each hospital training alone?
2. **Cost.** How much communication and training time does federation add?
3. **Fairness.** Does the shared model work equally well at every hospital?
4. **Deployment.** Do the results hold when each hospital runs as a separate, isolated system?

Data and simulated hospitals

We use the Diabetes 130-US Hospitals for Years 1999-2008 dataset (UCI, CC BY 4.0), about 100,000 inpatient encounters. The task is to predict readmission within 30 days; about 11% of encounters are positive.

Choice of dataset. The suggested datasets were LEAF (FEMNIST, Sent140), Synthea split by simulated hospital, and CIFAR-10 with Dirichlet partitioning. We substituted the UCI Diabetes dataset because it consists of real, de-identified hospital encounters with a clinically meaningful prediction task, which matches the hospital case study more closely than image or text benchmarks, while remaining public and openly licensed. Like Synthea, it has no hospital identifier, so we simulate sites, using a Dirichlet draw as suggested for CIFAR-10. The trade-off against Synthea is that the records are real rather than synthetic, though fully de-identified.

Cleaning and encoding. We drop weight (97% missing) and payer code (40% missing), and remove encounters ending in death or hospice. Diagnosis codes are grouped into broad ICD-9 categories. One-hot encoding and scaling are fitted on the training pool only, giving 182 features. 20% of patients, not encounters, are held out as a global test set of 19,773 rows, so no patient appears in both training and test.

Simulating hospitals. Whole patients are assigned to five hospitals. Patients are grouped by age band and readmission label, and a Dirichlet draw ($\alpha = 1.0$) sets each group's share per hospital, creating differences in age mix, readmission rate and size. Draws are rejected if a hospital holds under 5% or over 45% of the data, has under 200 positives, or has a positive rate outside 4% to 25%. Each hospital holds out 10% of its own patients for validation.

Hospital	Train rows	Validation rows	Readmission rate	Age under 50	Age 50 to 69	Age 70+
0	12,320	1,266	22.4%	5.7%	54.6%	39.8%
1	21,571	2,394	6.8%	14.7%	31.6%	53.7%
2	9,162	1,111	24.8%	14.9%	45.3%	39.8%
3	8,173	853	8.4%	13.0%	75.7%	11.3%
4	20,496	2,221	4.8%	25.0%	21.2%	53.8%

Table 1. The five simulated hospitals (71,722 training rows in total).

A side effect of the partition. Because age band and readmission label are assigned to hospitals separately, age becomes an artificial predictor of readmission inside some hospitals. In hospital 3, only 0.7% of patients aged 50 to 69 are readmitted. To keep this from inflating per-site results, we also report an age-adjusted ROC-AUC (see Evaluation method).

Model and training setup

All three approaches train the same model with the same hyperparameters and budget, so differences come from how the data is used.

Model. A PyTorch multilayer perceptron, 182 → 64 → 32 → 1, with ReLU and dropout 0.2 (13,825 parameters). The loss is binary cross-entropy with a positive-class weight (negatives divided by positives). We use Adam with learning rate 0.001 and batch size 64.

Centralized. The model trains on the union of all hospitals' training splits for 8 epochs. The epoch count was chosen on the pooled validation sets: validation ROC-AUC peaked at 0.672 after 8 epochs and fell to 0.635 after 30.

Federated. A Flower 1.42 ServerApp and ClientApp. Each round, the server sends the global

weights to all five hospitals; each trains for one local epoch and returns its weights and sample count; the server combines them with FedAvg, a weighted average by sample count. We run 8 rounds of 1 local epoch, matching the centralized budget, from identical initial weights. Each hospital computes its class weight from its own data, and the optimizer is rebuilt every round, both standard in FedAvg.

Local-only. Each hospital trains the same model on its own data for the same budget and never communicates.

All experiments use five seeds (42 to 46) on the same machine, a MacBook with Apple Silicon, so timings are comparable.

System architecture

The same Flower application runs in two modes with no change to the model, training or aggregation code.

Single-process mode. Flower's simulation engine, built on Ray, runs the server and five clients as worker processes on one machine. It is quick to iterate on, but the separation between hospitals exists only in code.

Container deployment. Docker Compose with Flower's deployment engine (Figure 1). The server container runs a SuperLink, which coordinates rounds, and the ServerApp, which performs FedAvg and evaluates the global model on the test set. Each hospital runs a SuperNode and its ClientApp in its own container. One command, python docker/run_experiment.py, starts the stack, checks isolation, runs all seeds and records traffic, time and memory.

Figure 1. Container deployment: one server container and five hospital containers. Only model weights and metrics travel. [Insert the architecture diagram from the README]

Isolation by construction. Each hospital container mounts only its own data file, read-only, and sits on a private network shared only with the server. The server holds no hospital data. Five automated checks confirm this on the running containers and are saved in results/docker/isolation_check.json.

Design decisions.

- Ubuntu-based Flower images, because the default Alpine images use the musl C library, for which PyTorch publishes no packages.
- CPU-only PyTorch, avoiding several gigabytes of GPU libraries the containers cannot use.
- One CPU per hospital and two for the server, approximating separate machines of modest size.
- A lean client. Replacing scikit-learn and pandas metric code with an equivalent NumPy-only version, verified identical, cut each client process from 463 MB to 359 MB, so the stack fits in the 3.8 GiB available to Docker.

Evaluation method

Headline metric. ROC-AUC on the global test set, which measures ranking and does not depend on a threshold. It varies by only 0.001 to 0.002 across seeds. Recall at the 0.5 threshold varies by 0.03 to 0.05, so we report it but draw no conclusions from small changes.

Per-site performance and fairness gap. Each hospital's ROC-AUC on its own validation set, with 95% bootstrap intervals. The fairness gap is the maximum minus the minimum per-site

ROC-AUC. Validation sets are small; hospital 3 has only 56 positive cases.
Age-adjusted ROC-AUC. ROC-AUC computed only over pairs of patients in the same age band. It removes the artificial age signal created by the partition.
Paired bootstrap test. Each draw resamples patients once and scores both models on the same rows; we average over five seeds with 1,000 draws, and apply a Bonferroni correction across the five hospitals. Reloaded models reproduce their recorded ROC-AUC to about 10^{-8}.
Cost. Bytes of model weights exchanged, plus real traffic at the server's network interface in the deployment; and wall-clock training time from the first round to the last.

Results

Accuracy and cost

Measure (5 seeds)	Centralized	Federated	Difference
Global ROC-AUC	0.6777 ± 0.0023	0.6696 ± 0.0012	−0.008 (95% CI −0.013 to −0.003, p = 0.001)
Global recall	0.623 ± 0.040	0.641 ± 0.056	+0.018 (threshold shift)
Per-site ROC-AUC gap	0.080	0.090	+0.010 (p = 0.75)
Training time	5.8 s	34 ± 8 s	about 6× slower
Communication	none	4.5 MB	6.7 MB including per-site evaluation

Table 2. Centralized against federated training in single-process mode, on the same machine.
The accuracy loss is about 3.5 times the seed noise, so it is real but small. Federated ROC-AUC levels off at about 0.67 by round 3 (Figure 2), so more rounds alone are unlikely to close the gap. Communication matches the estimate of 13,825 float32 parameters, 10 transfers per round, over 8 rounds. Most of the extra time is framework overhead: round 1 alone takes about 13 s of summed client time because workers start cold, against about 1.5 s for later rounds.
Figure 2. Global ROC-AUC per round. [Insert results/figures/comparison_convergence.png]

Per-hospital fairness

Hospital	Readmission rate	Change in ROC-AUC, federated minus centralized	Significant after correction
0	22.4%	−0.016	No
1	6.8%	no detectable change	No
2	24.8%	−0.025 (95% CI −0.040 to −0.010, p = 0.001)	Yes
3	8.4%	+0.025	No (56 positive cases)
4	4.8%	no detectable change	No

Table 3. Paired comparison per hospital, on each hospital's validation set.
Only hospital 2 is significantly worse under federation, and the loss remains after age adjustment. The overall gap rises from 0.080 to 0.090, but the difference is not significant (p = 0.75), so we cannot say federation made fairness better or worse.
Figure 3. Per-hospital ROC-AUC with 95% intervals. [Insert

results/figures/comparison_per_site.png]

Local-only training

The local-only baseline answers the question each hospital would ask: is joining worth it? On a hospital's own validation patients, training alone looks strong for some sites before adjustment; hospital 3's local model scores 0.889, but this comes from the artificial age signal. Once age is adjusted for, no hospital does better alone than in the federation. On patients from all hospitals, the federated model beats every local model by 0.005 to 0.113 AUC, all with $p \leq 0.05$. A hospital's own model learns its own population and transfers poorly to others; the federated model does not.

Figure 4. Local-only, federated and centralized ROC-AUC per hospital. [Insert results/figures/local_only_comparison.png]

Container deployment against single-process mode

Measure (5 seeds)	Single-process	Docker (6 containers)
Global ROC-AUC	0.6696 ± 0.0012	0.6699 ± 0.0012
Per-site ROC-AUC gap	0.090 ± 0.004	0.089 ± 0.002
Communication, 8 rounds	6.7 MB (estimated)	12.3 ± 0.1 MB (measured)
Training time in the app	34 ± 8 s	116 ± 20 s
End to end	not recorded	217 ± 42 s

Table 4. The same federated training as one process and as isolated containers.

Per seed, the two modes agree to within 0.0006 ROC-AUC. The deployment sends about 1.8 times the estimated traffic and is about 3.4 times slower (Figure 5); the next section explains both.

Figure 5. Time per round in the container deployment. [Insert results/figures/docker_round_seconds.png]

Discussion

A small privacy cost with a clear gain. Federation costs 0.008 AUC against pooling all data, a tenth of the 0.08 gap between the easiest and hardest hospital. Against training alone, every hospital's model generalizes better inside the federation. For a hospital that cannot share records, joining is the better option.

Who pays the cost. The two hospitals whose AUC fell, 0 and 2, have the highest readmission rates (22.4% and 24.8%) and together hold 30% of the training data, while low-rate hospitals 1 and 4 hold 59%. FedAvg weights updates by sample count, so the global model leans toward low-readmission populations, and local class weights that differ about sixfold across sites may add to this. These are hypotheses consistent with the pattern; we have not tested them.

Communication in practice. The deployment sent 12.3 MB rather than the 6.7 MB the model size predicts. Round 1 ships the application code to each hospital (about 240 KB each), and every round adds HTTP overhead, polling and heartbeats (2.7 KB/s even when idle). An estimate from model size alone understates real traffic by almost half.

Time. With a model this small, coordination costs far more than computation. In the deployment, Flower starts a fresh client process for every message, each imports PyTorch, and each hospital

has one CPU. The balance would shift for larger models.

Responsible AI

Privacy. Patient records never leave a hospital, and in the deployment this is enforced and verified rather than assumed. Federated learning alone is not a formal guarantee: shared weights can leak information through membership-inference or reconstruction attacks, and we use neither differential privacy nor secure aggregation.
Fairness. A single global metric hid the most important finding, hospital 2's loss of 0.025 AUC. Any hospital joining a federation should see its own results before deployment. Our analysis covers fairness across sites only, not across patient groups such as sex or race. The age artifact also shows that per-site metrics can mislead when site populations are built unevenly, which is why we report the age-adjusted version.
Transparency and reproducibility. The repository includes a data card, a model card, fixed seeds and one-command pipelines. Repeated runs are close but not bit-identical (seed 42 gave 0.6681 to 0.6689), because parallel workers sum floating-point numbers in varying order, so we report means over five seeds.
Safety. A ROC-AUC near 0.67 is typical for this dataset but too weak for automated clinical decisions. Any use should be as a decision-support signal reviewed by clinicians.

Limitations and future work

- **Simulated sites.** All hospitals come from one dataset, and the partition creates an artificial link between age and readmission inside some sites. A partition that assigns age and label jointly would remove it.
- **One machine.** Both modes ran on one laptop, so network conditions are far better than between real hospitals, and timing was affected by other load.
- **Small validation sets.** Per-site estimates are noisy, especially for hospital 3.
- **No formal privacy guarantee.** No differential privacy or secure aggregation.
- **Old data.** US hospitals, 1999 to 2008.

Next steps are a shared global class weight and FedProx to test the cause of hospital 2's loss; sweeps over rounds, local epochs and non-IID severity; differential privacy on client updates; and a leave-one-hospital-out test of transfer to a new site.

Conclusion

Federated learning let five hospitals with different patient populations train a shared readmission model without moving patient records, at a cost of 0.008 ROC-AUC, about 6× the training time and a few megabytes of communication. Every hospital's model generalizes better inside the federation than alone, and the gap between the best and worst hospital did not change significantly. One high-readmission hospital lost 0.025 AUC, which the global average concealed. Running each hospital as an isolated container reproduced these results while verifying data isolation and measuring real network cost. Per-site reporting should be standard when evaluating any federated model.

Team contributions

Member	Contribution
Sunidhi	Data cleaning, patient-grouped split and encoding; simulation of the five hospitals; shared model and metrics; centralized baseline and epoch selection; data card
Sonia Sharma	Flower server and clients; federated runs over five seeds; communication and timing measurement; comparison notebook and paired bootstrap tests; local-only baseline and age-adjusted analysis; Docker deployment and isolation checks

References

1. Strack, B., DeShazo, J. P., Gennings, C., et al. (2014). Impact of HbA1c measurement on hospital readmission rates: analysis of 70,000 clinical database patient records. BioMed Research International.
2. UCI Machine Learning Repository. Diabetes 130-US Hospitals for Years 1999-2008.
3. McMahan, B., Moore, E., Ramage, D., Hampson, S., and Agüera y Arcas, B. (2017). Communication-efficient learning of deep networks from decentralized data. AISTATS.
4. Beutel, D. J., Topal, T., Mathur, A., et al. (2020). Flower: A friendly federated learning research framework. arXiv:2007.14390.
5. Hsu, T.-M. H., Qi, H., and Brown, M. (2019). Measuring the effects of non-identical data distribution for federated visual classification. arXiv:1909.06335.
6. Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., and Smith, V. (2020). Federated optimization in heterogeneous networks. MLSys.